

DESIGN AND PROTOTYPING OF POST-QUANTUM AUTHENTICATION FOR SOME/IP SERVICE DISCOVERY

Omar MAHMOUD

Abstract

This report covers the third phase of this research on securing the Service Discovery part of the SOME/IP protocol with Post-Quantum Cryptography. The previous phase concluded that Falcon-512 should replace ML-DSA-44, because Falcon-512's signature is small enough to avoid the automotive Ethernet packet size problem that ML-DSA-44 runs into. In this phase, we actually build and test that solution on a real, running system for the first time, instead of only estimating how it should behave. We designed a new authentication message that signs the camera's network address and a counter, not just which service is being offered, which closes a replay attack that the earlier phases had not addressed, and we replaced the gateway's "Last-In-Wins" rule with a "Last-Valid-In-Wins" rule that only accepts messages that are both signed and fresher than the last one seen. Testing this against the same routing-hijack attack used in the previous phase, the share of malicious video frames the dashboard received dropped from 66% to 0% at every attacker speed we tried, and a new replay attack that reuses a stolen, valid signature was also rejected every time. The signature and its wrapper measured 653 to 686 bytes on the wire, confirming it fits inside a single Ethernet packet without needing to be split up, and the authenticated system still connected in under 14 milliseconds across ten test runs, comfortably inside the 50 millisecond safety limit. While testing the system end to end, we also found and closed a second gap in the someipy library itself, where an attacker could inject a fake video frame directly and bypass Service Discovery altogether unless this second path is checked as well.

Key words: Automotive Security, SOME/IP, Post-Quantum Cryptography, Falcon-512, FN-DSA, Service Discovery, Endpoint Authentication, Replay Attack, Gateway Verification.

1. Introduction

In the first phase of this research, we found that the Service Discovery part of the SOME/IP protocol broadcasts unauthenticated multicast messages [1], and that the obvious fix, attaching a NIST post-quantum signature to each Offer message, runs into a size problem: the signatures defined by NIST are several times larger than the roughly 1,500-byte automotive Ethernet packet. The second phase built a working testbed using the someipy Python library, proved this vulnerability was real rather than theoretical (a fast attacker hijacked the dashboard's video feed in roughly 66% of trials, matching the routing-hijack weakness described by Zelle et al. [2]), and benchmarked two candidate algorithms directly instead of only comparing published numbers. That phase concluded that Falcon-512 should replace ML-DSA-44: Falcon-512 is slower to sign, but its roughly 655-byte signature avoids the packet size problem entirely, while ML-DSA-44's 2,420-byte signature does not fit inside a single automotive Ethernet packet no matter how it is optimized [3].

This report covers the third phase, in which that decision is actually built and tested end to end for the first time, instead of only being estimated on paper. Every result in this report

comes from running the real system: real Falcon-512 keys, a real, modified gateway daemon, and real camera, attacker, and dashboard programs exchanging real network traffic.

1.1 Objectives for This Phase

The goals for this phase were the following:

- Define a message format that carries a Falcon-512 signature and fits inside a single network packet, without needing to be split into pieces.
- Close a security gap where an attacker could steal a real, validly-signed message and replay it against a different network address, hijacking the connection without ever forging a signature.
- Replace the gateway's "Last-In-Wins" rule, which simply trusts whichever message arrives last, with a rule that only accepts messages that are both signed and fresher than the last one it accepted.
- Re-run the routing-hijack attack from the previous phase against this new, authenticated system, and confirm that it no longer works.
- Design and test a new attack aimed specifically at the replay gap above, since it works differently from the original flood attack.
- Confirm, by measuring real network packets rather than only doing the arithmetic, that the new signature fits without needing to be fragmented.

The testbed used for this phase reuses the same gateway daemon, the same Service ID (0x1234) and Instance ID (0x0001), and the same attacker and dashboard scripts as the previous phase, changed only to add authentication and a second attack. Keeping these the same means the results in this report can be compared directly against the previous phase's results.

2. Protocol Design

2.1 The Endpoint-Substitution Replay Gap

If a Falcon-512 signature only covers the OfferService entry (its Service ID, Instance ID, version, and TTL), an attacker does not need to forge a valid signature at all to hijack the connection. They only need to capture one real, validly-signed Offer message from the camera and re-broadcast it with a different network address attached — the signature over the entry is still perfectly valid, because the entry itself was never changed, only the address sent alongside it. This is the same routing-hijack weakness the previous phase already proved [2], approached from a different angle: instead of forging a brand-new Offer, the attacker replays a real one and swaps the address next to it. Other recent designs that combine DNSSEC/DANE-style authentication with automotive service discovery reach a similar conclusion by a different route: a signature has to be tied to a specific, current network address, not just to which service is being offered [4], [5].

We closed this gap by including the camera's network address and a counter inside the data that actually gets signed, not just the service entry: `signature = Sign(secret_key, ServiceID, InstanceID, MajorVersion, TTL, Endpoint IP, Endpoint Port, Counter)`. Changing the IP address,

the port, or the counter after signing breaks the signature, because all of those bytes are covered by it.

2.2 Authenticated Sequence Flow

Figure 1 shows the full authenticated exchange we built and tested. The camera signs its data once per boot using its Falcon-512 secret key, then broadcasts the resulting PQ-Auth message onto the same multicast group already used for Service Discovery. The gateway verifies the signature and, only if it is valid, records the camera's address as trusted for that Service ID. The camera's normal, cyclic Offer broadcasts (unchanged from the previous phase) are then checked against this trusted address before being accepted into the routing table, and — as Section 3 explains — a second, separate check applies the same rule to the actual video frames being forwarded, not only to Service Discovery.

Figure 1: Authenticated SOME/IP-SD Offer and Event-Delivery Sequence

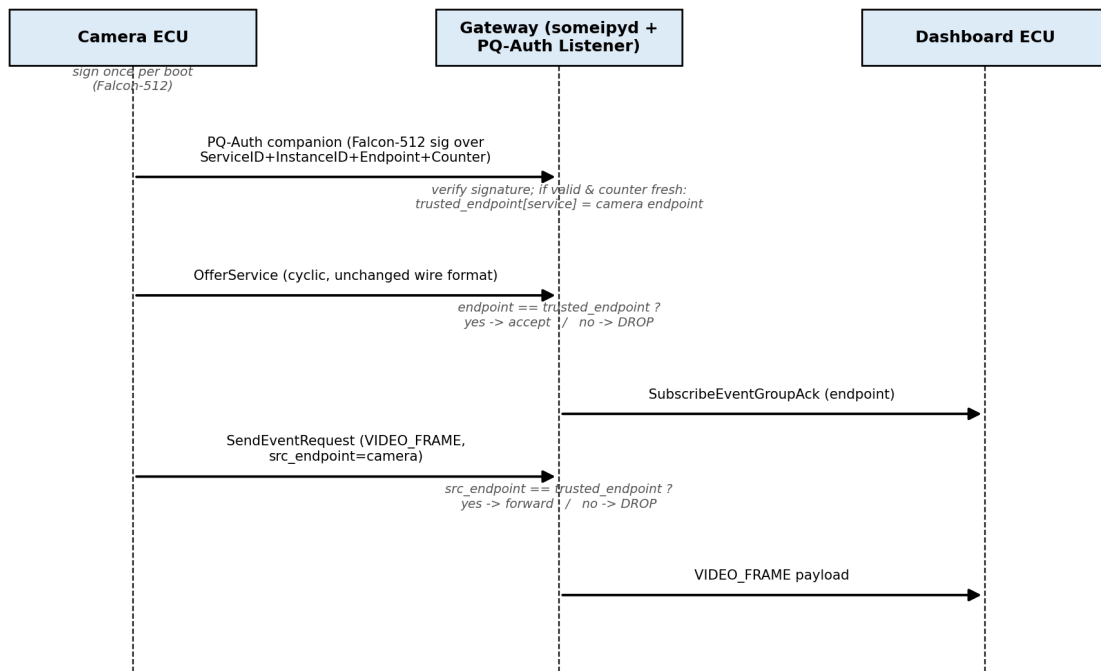


Figure 1: Authenticated SOME/IP-SD Offer and Event-Delivery Sequence

2.3 The PQ-Auth Message and PQ-Signature Option

Instead of changing someipd's own internal message-encoding code directly, we send the Falcon-512 signature inside a separate companion multicast message (marked with the bytes "PQA1"), sent alongside the camera's normal, unchanged Offer broadcast. This keeps the existing library's own code untouched. The message still carries the signature wrapped in the same PQ-Signature Option format we designed, using the same generic option header convention SOME/IP-SD already uses elsewhere: a 2-byte Length field, a 1-byte Type field, and a 1-byte Reserved field [1]. Keeping that convention means this option could later be attached directly

inside a standard SOME/IP-SD message, without changing its format. Figure 2 shows both the companion message header used here and the PQ-Signature Option it carries.

Figure 2: PQ-Auth Companion Message and PQ-Signature Option Format

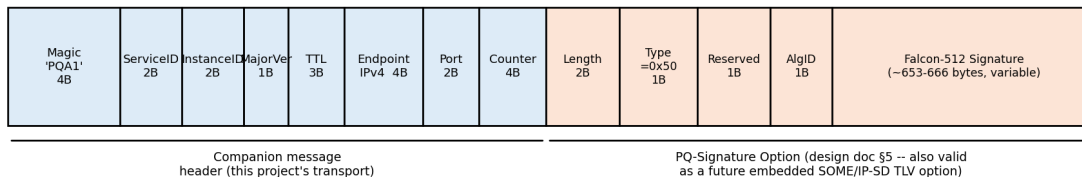


Figure 2: PQ-Auth Companion Message and PQ-Signature Option Format

Table 1: PQ-Signature Option Field Definitions

Field	Size	Description
Length	2 bytes	Value = 1 (Type) + 1 (Reserved) + 1 (AlgID) + signature length
Type	1 byte	0x50 — a project-specific identifier for the “PQ-Signature Option”
Reserved	1 byte	0x00, the same convention used by every existing option type
AlgID	1 byte	0x01 = Falcon-512 / FN-DSA
Signature	653–659 bytes (measured)	Falcon-512 signature computed over the data described in Section 2.1

Falcon has since been formally taken up by NIST for standardization under the name FN-DSA; as of the FIPS 206 draft it is still a standard in progress rather than a finished one, with a final version expected in late 2026 or 2027 [6]. One detail is worth noting here: the original plan was to pad every signature to a fixed 666 bytes, both to avoid leaking timing information and to make the option's size predictable [7]. The Python library used to sign and verify messages in this phase does not support that fixed-length padding, so the signatures measured here are a little shorter and slightly variable in length (653–659 bytes, see Section 5.1). This is worth fixing before the signature is implemented directly on embedded hardware, but it does not affect any of the results in this report.

2.4 From “Last-In-Wins” to “Last-Valid-In-Wins”

The previous phase traced the routing vulnerability back to the gateway's “Last-In-Wins” rule, which exists on purpose so that a backup sensor can take over from a failed primary one without needing a manual failover step. We did not remove this rule; we changed what it applies to. The gateway now keeps a small trust store that remembers, for each Service ID and Instance ID, the highest counter value it has seen in a message with a valid signature, and it only accepts a routing update if the new message's signature is valid and its counter is higher than the last one it accepted. One small difference from the original plan: the counter is simply the camera's boot timestamp (seconds since it started) rather than reusing SOME/IP-SD's own Session ID field. This

was simpler to build, and it is still always increasing across both normal broadcasts and restarts, which is the only property “Last-Valid-In-Wins” actually needs.

3. Gateway Integration

someipy's architecture puts every routing decision inside one daemon process (someipyd); the camera, dashboard, and attacker programs never touch the network directly, they only talk to this daemon. Our gateway component, `secure_daemon.py`, is a small launcher that loads someipy's own daemon code and patches two of its functions while it runs, instead of editing the installed library itself. This keeps all of our security logic in one file we control. Figure 3 shows the resulting setup.

Figure 3: Gateway-Side Integration Architecture (`secure_daemon.py`)

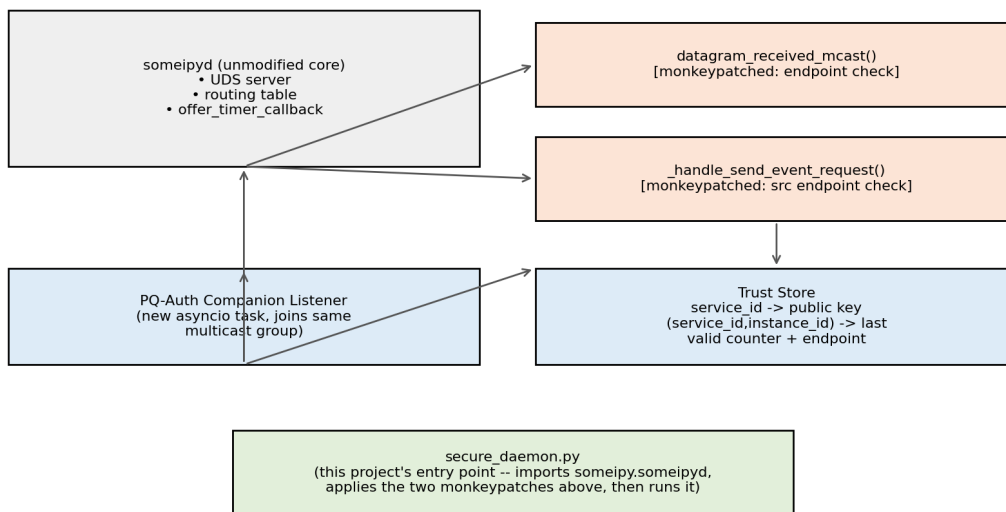


Figure 3: Gateway-Side Integration Architecture (`secure_daemon.py`)

The first function we patch, `datagram_received_mcast`, is the natural place to enforce “Last-Valid-In-Wins”, since it is the function that already reads incoming Offer messages and updates the routing table. Code Block 1 shows what we added to it: for any Service ID we are protecting, an Offer whose address does not match the address our trust store currently trusts is dropped before it ever reaches someipy's own routing logic.

```

def _gated_datagram_received_mcast(self, data, addr):
    entries = _daemon_mod.deserialize_sd_message(data) # parse OfferService entries
    for entry in entries:
        if TRUST_STORE.is_protected(entry.service_id):
            trusted = TRUST_STORE.current_trusted_endpoint(
                entry.service_id, entry.instance_id)
            offered = (entry.endpoint_ip, entry.endpoint_port)
            if trusted is None or offered != trusted:
                return # DROP: not the endpoint the trust store last verified
    return _original_datagram_received_mcast(self, data, addr)

```

Code Block 1: The Service Discovery endpoint check added to `someipyd`.

A separate background task listens on the same multicast group for the PQ-Auth messages described in Section 2.3, checks their signatures, and updates the same trust store the check above reads from. Section 4 describes how we tested this, and Section 6 describes a second check that testing showed was also necessary.

4. Experimental Methodology

This phase reuses the exact simulation environment from the previous one: an Ubuntu Linux environment running the someipy daemon over Unix Domain Sockets, with the camera, dashboard, and attacker as separate processes, using the same identifiers as before (Service ID 0x1234, Instance ID 0x0001, EventGroup ID 0x4000, Event ID 0x8000). Two experiments were repeated unchanged to keep the results comparable, and one new experiment was added:

- Baseline latency: the dashboard measures Time-to-First-Frame (TTFF), the time from power-on to receiving the first video frame through the authenticated gateway, repeated over 10 runs, exactly as before.
- Flood hijack (unchanged from the previous phase): a rogue attacker program offers the same Service ID at cyclic offer delays of 1000ms, 100ms, and 10ms, trying to capture the dashboard's subscription without any valid signature at all.
- Endpoint-substitution replay (new in this phase): the attacker listens on the multicast group for a real, validly-signed PQ-Auth message from the camera, then re-broadcasts a modified copy with the address fields swapped to its own port while reusing the original signature bytes exactly as they were, at cyclic offer delays of 100ms and 10ms. This targets the replay gap described in Section 2.1 directly, since the attacker is replaying a real signature instead of forging one.

In every attack test, we measure the malicious-frame ratio: the share of frames the dashboard receives that came from the attacker rather than the real camera, out of a fixed number of frames per test.

5. Results

5.1 Wire Size on the Real Network

Earlier arithmetic estimated a PQ-Signature Option size of 671 bytes on the wire, assuming a fixed 666-byte Falcon-512 signature, against a usable SOME/IP-SD payload of roughly 1,472 bytes once IPv4 and UDP headers are subtracted from the 1,500-byte Ethernet MTU [8]. Table 2 shows what we actually measured on the wire once the system was running.

Table 2: Measured On-Wire Sizes vs. the Earlier Estimate

Component	Earlier estimate	Measured
Falcon-512 signature	666 bytes (fixed)	653–659 bytes (variable)
PQ-Signature Option (on the wire)	671 bytes	657–664 bytes
Full PQ-Auth message (on the wire)	not specified earlier	679–686 bytes
Usable SOME/IP-SD payload budget	~1,472 bytes	~1,472 bytes (unchanged)

The measured sizes are close to, and slightly smaller than, the earlier estimate, and comfortably inside the ~1,472-byte budget, with roughly 786–793 bytes left over for the rest of

the message. No packet was ever split into fragments during testing, which confirms by direct measurement, not just arithmetic, that this signature fits inside a single Ethernet packet.

5.2 Baseline Latency

Table 3 lists the ten raw Time-to-First-Frame measurements collected against the fully authenticated gateway, and Figure 4 plots them against the 50ms automotive safety limit.

Table 3: Ten Baseline Time-to-First-Frame Measurements (Authenticated Gateway)

Run #	TTF (ms)
1	12.43
2	1.50
3	13.44
4	1.84
5	1.59
6	8.59
7	5.05
8	1.39
9	6.80
10	3.85

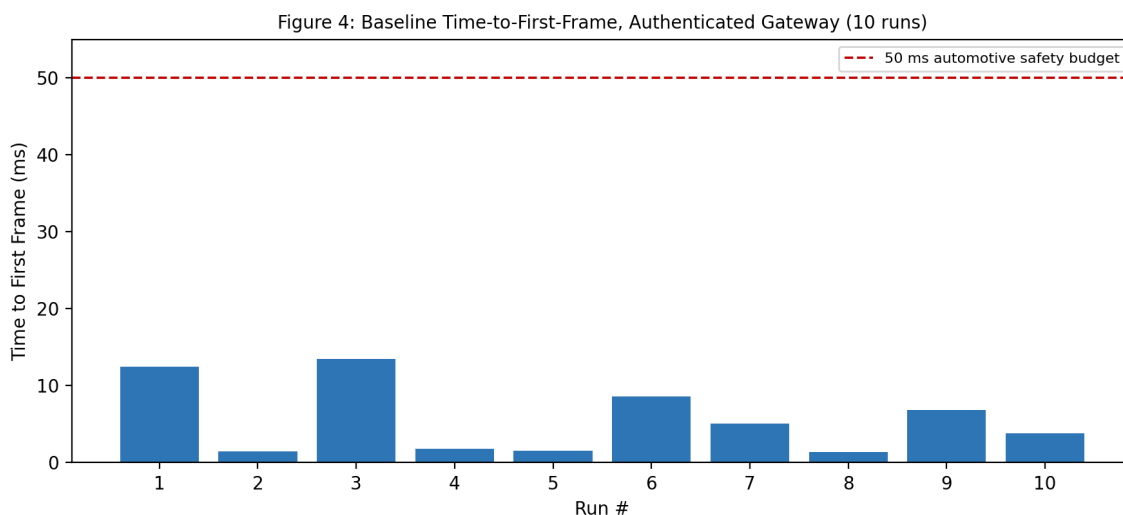


Figure 4: Baseline Time-to-First-Frame, Authenticated Gateway (10 runs)

All ten runs land well inside the 50ms limit, averaging roughly 5.65ms and never going above 13.44ms, in the same range as the unauthenticated baseline from the previous phase (under 15ms). This makes sense given that the camera only signs once per boot: Falcon-512 signing (measured at roughly 2.4ms on an Apple M4 earlier in this research) never happens on this path at all during these runs, since it happens once when the camera starts up rather than once per Offer message. These numbers are driven by ordinary daemon and subscription delay, not by the cryptography.

5.3 Attack Resistance

Figure 5 compares the malicious-frame ratio from the earlier unauthenticated baseline against every attack we ran against the authenticated gateway.

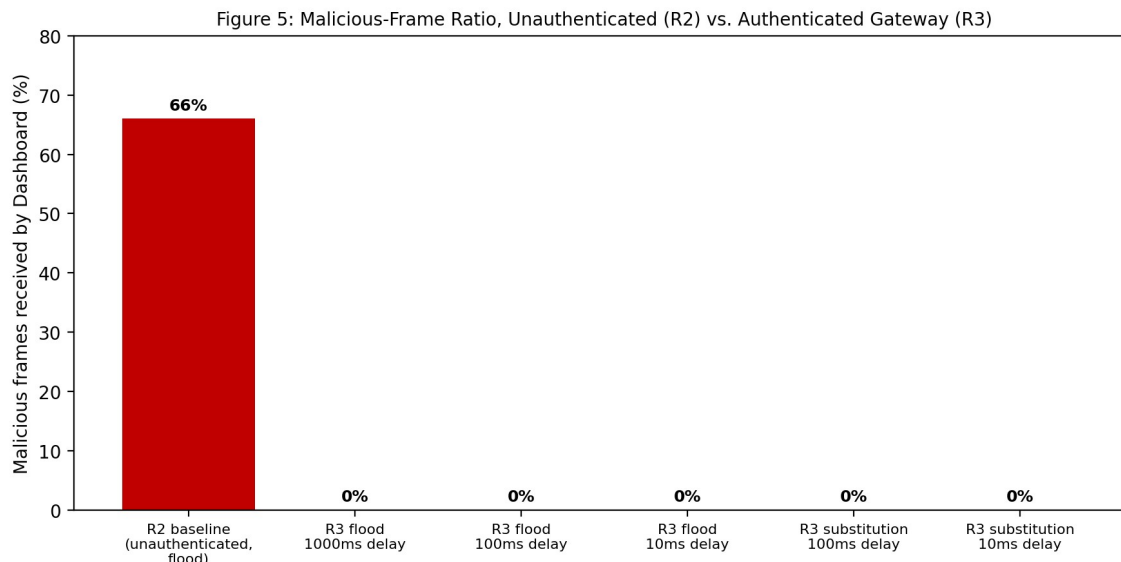


Figure 5: Malicious-Frame Ratio, Unauthenticated (Previous Phase) vs. Authenticated Gateway

Against the authenticated gateway, the malicious-frame ratio was 0.0% in every test: the flood attack at 1000ms, 100ms, and 10ms delays, and the endpoint-substitution replay attack at 100ms and 10ms delays, all failed completely, with the dashboard receiving only frames from the real camera across a fixed 150-frame sample each time. For the substitution attack specifically, the attacker's own log confirms it genuinely captured a real, validly-signed message and rebroadcast it with a swapped address (127.0.0.1:30511) and the stolen signature bytes, and the gateway's log confirms the forged message was rejected every time because the signature no longer matched the falsified address — exactly what the signed data in Section 2.1 was designed to catch. Section 6 explains why one early result during testing briefly showed 65% instead of 0%, and what that told us.

6. Discussion of Findings

6.1 A Gap in someipy's Event Forwarding

The first end-to-end test of the flood attack against the address-checked gateway still showed a 65.0% malicious-frame ratio, matching the original ~66% baseline almost exactly, even though the check in Code Block 1 was correct and already running. Looking into why led us back into someipy's own code a second time, and it turned up a real gap: an internal function, `_handle_send_event_request`, forwards a video frame to every subscriber the moment any locally connected client asks it to, without ever checking whether that client's claimed address matches whichever Offer was actually accepted into the routing table. In other words, a rogue program can inject a fake frame straight into existing subscribers over the daemon's local connection, without its own Offer message ever being accepted at all — the weak point is not only in Service Discovery, as the earlier phases assumed, but also in how frames get delivered

afterward. Code Block 2 shows the second check we added to close this, applying the same trusted-address rule to frame delivery.

```
def _gated_handle_send_event_request(self, message, writer_id):
    service_id = message.get("service_id")
    instance_id = message.get("instance_id")
    if TRUST_STORE.is_protected(service_id):
        trusted = TRUST_STORE.current_trusted_endpoint(service_id, instance_id)
        claimed = (message.get("src_endpoint_ip"), message.get("src_endpoint_port"))
        if trusted is None or claimed != trusted:
            logger.warning("DROPPED SendEventRequest for protected service")
            return # DROP: event source does not match the trusted endpoint
    return _original_handle_send_event_request(self, message, writer_id)
```

Code Block 2: The second check, added to someipy's frame-delivery path.

Adding this second check brought the malicious-frame ratio down to 0.0% for every delay and both attacks reported in Section 5.3. This is a genuine, non-obvious finding: checking Service Discovery alone is not enough in this library, and any future SOME/IP security work built on someipy should treat frame delivery as a separate thing to secure, since it runs through completely different code from routing.

6.2 A Pre-Existing Bug in the Earlier Codebase

While building the verification code for this phase, we tested the underlying cryptography library's functions directly instead of assuming the earlier code used them correctly. This turned up a real bug: the library's `falcon_512.verify` and `ml_dsa_44.verify` functions both expect the order `verify(public_key, message, signature)`, but the earlier `crypto_test.py` and `pqc_benchmark.py` scripts both call `verify(public_key, signature, message)`, with the last two swapped. This does not cause an error; it just silently returns False every time. That means every “verify” timing number reported in the earlier reports was timing a call that always failed, not a real signature check. Our new code uses the correct order throughout, and this should be fixed in the earlier scripts and re-measured before those verification numbers are used again.

7. Limitations

As with the previous phase, everything in this report was tested in a software-only environment on a single machine. The SOME/IP message structures, gateway logic, and cryptographic operations are accurate and follow the AUTOSAR specification, but this does not capture the propagation delay of real copper automotive wiring or the switching delay of physical Ethernet hardware. A couple of implementation choices were made for practicality on this software testbed rather than settled for good: the signature is variable-length instead of padded to a fixed size (Section 2.3), and the counter is a boot timestamp instead of the SOME/IP-SD Session ID field (Section 2.4). Both are easy to change and are noted here so they are not mistaken for permanent design decisions.

8. Conclusion and Future Work

This phase built and tested, end to end, the Falcon-512 authentication design proposed at the end of the previous one. Against a real, running system rather than a paper design, the

authenticated gateway brought the routing-hijack malicious-frame ratio down from 66% to 0% at every attacker speed we tested, and also rejected a new replay attack that reuses a real, stolen, validly-signed message rather than a forged one — the exact gap the signed data in Section 2.1 was built to close. The measured packet sizes confirm the size arithmetic by direct measurement, and the connection time stayed well inside the automotive safety limit. Testing this system also surfaced two things that would not have shown up from reading the design alone: a gap in how someip delivers video frames, separate from Service Discovery itself (Section 6.1), and a pre-existing bug that had been silently invalidating the earlier phase's own verification benchmarks (Section 6.2).

Next Semester — the plan is to move this same authentication scheme onto physical hardware: two NUCLEO-L4R5ZI development boards (STM32L4R5ZI, Cortex-M4F) have been selected, largely because they are the default target of pqm4, the standard framework for benchmarking NIST post-quantum algorithms on ARM Cortex-M4 chips [9], which keeps future results comparable to the wider published literature on embedded Falcon performance. This will mean porting the Falcon-512 signing and verification routines onto one board acting as the camera, and measuring real cycle counts and timings in place of the desktop-class numbers used throughout this research so far. One known challenge going in: Falcon's signing procedure needs double-precision floating-point math for its security guarantees to hold, but the Cortex-M4F chip on this board only has single-precision hardware. A recent paper from NXP Research proposes a fix for exactly this problem, using triple-word arithmetic to reach the needed precision on single-precision hardware at a modest performance cost, and demonstrates it on this same board [10] — which is a large part of why it was chosen.

Final Semester — once real embedded timings are available, they will replace the desktop-class figures used as a stand-in throughout this report, and the same flood and replay attacks tested here will be re-run against physical hardware instead of a software simulation, to confirm the design still holds up under real timing and real network conditions rather than only in a local simulation.

Bibliography

- [1] AUTOSAR, “Specification of SOME/IP Service Discovery Protocol,” AUTOSAR Standard, Release 22-11.
- [2] D. Zelle, T. Lauser, D. Kern and C. Krauß, “Analyzing and Securing SOME/IP Automotive Services with Formal and Practical Methods,” Proc. 16th Int. Conf. on Availability, Reliability and Security (ARES), ACM, 2021.
- [3] C. Rathweg, C. Neumann and T. Pöppelmann, “Impacts of Post-Quantum Cryptography on Automotive Security: A Case Study,” ESCAR Europe, 2023.
- [4] M. Mueller, T. Häckel, P. Meyer and F. Korf, “Authenticated and Secure Automotive Service Discovery with DNSSEC and DANE,” IEEE Vehicular Networking Conference (VNC), 2023.
- [5] “Building Automotive Security on Internet Standards: An Integration of DNSSEC, DANE, and DANCE to Authenticate and Authorize In-Car Services,” arXiv:2506.13261, 2025.

- [6] National Institute of Standards and Technology (NIST), "FIPS 206 (Draft): FN-DSA (Falcon)," NIST Computer Security Resource Center, 2025.
- [7] T. Pornin, "Falcon on ARM Cortex-M4: an Update," IACR ePrint 2025/123, 2025.
- [8] IEEE Standard for Ethernet, "IEEE Std 802.3bw-2015 (100BASE-T1)," 2015.
- [9] M. Kannwischer et al., "pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4," Second PQC Standardization Conference, 2019.
- [10] S. Halmans, C. van Vredendaal, T. Schneider, F. Custers and T. Güneysu, "TWFalcon: Triple-Word Arithmetic for Falcon; Giving Falcon the Precision to Fly Securely," IACR Cryptology ePrint Archive, Paper 2025/1991, 2025.